ENTERPRISE DATA RECONCILIATION

Financial Audit and Revenue Analysis Using Python, SQL and Data Visualization

Project type	Data Analytics / Financial Reconciliation
Reporting period	1 January – 31 December 2023
Source systems	SQLite user database, server logs, daily EUR/USD rates
Tools	Python, Pandas, SQLite, Regular Expressions, Matplotlib

Project Report

Executive Summary

This project builds a repeatable reconciliation pipeline across three separate data sources: historical customer account records in SQLite, unstructured transaction server logs, and daily EUR-to-USD exchange rates. The pipeline identifies each customer's latest status, excludes failed log events, extracts transaction fields with regular expressions, fills missing daily exchange rates, and calculates reconciled revenue for transactions associated with currently active users.

Measure	Result
Raw server log records	100,000
Error records excluded	5,098
Valid transactions parsed	94,902
Latest active users	14,183
Missing exchange rates forward-filled	104
Transactions reconciled to active users	53,899
Active customers with transactions	13,888
Total reconciled USD revenue	$148,274,622.91
Pipeline processing time (seconds)	1.952

These are computed results from the supplied data files. The reported total includes only successfully parsed transactions matched to users whose latest database status is Active.

1. Introduction

Enterprise systems often store information in different formats and maintain historical versions of the same record. Transaction logs may be unstructured text, customer databases may contain repeated status updates, and financial rates may be unavailable on non-trading days. This project demonstrates how to transform these sources into a consistent transaction dataset for financial analysis.

2. Problem Statement

The objective is to reconcile transaction records with the correct customer status and daily currency conversion rate without silently losing valid transaction rows during rate matching. The pipeline must be efficient for 100,000 log records and produce monthly revenue and top-customer visualizations.

3. Objectives

• Retrieve each user's latest status and retain only Active users.

• Exclude server log entries marked ERROR.

• Use regular expressions to extract date, user ID, product ID and EUR transaction amount.

• Forward-fill missing exchange rates in chronological order.

• Calculate USD revenue and create an audit-ready final transaction table.

• Summarize monthly revenue and identify the five highest-value active customers.

4. Data Sources

File / table	Description	Processing role
enterprise_database.db / users	47,464 historical account records across 25,000 distinct users	SQL window function selects latest status
server_logs.txt	100,000 server log records	Filter ERROR and extract fields with regex
daily_exchange_rates.csv	365 dates in 2023, including 104 missing rate values	Chronological forward-fill for missing rates

5. Methodology and Implementation

Phase 1 — SQL extraction

The query uses ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY updated_at DESC) to rank each user's historical rows. The row ranked 1 is the latest status. Only after that selection is status filtered to Active. This prevents an old Active row from incorrectly qualifying a user whose latest status is Suspended or another status.

```
WITH ranked_users AS (
  SELECT user_id, name, status, updated_at,
         ROW_NUMBER() OVER (
           PARTITION BY user_id ORDER BY updated_at DESC
         ) AS rn
  FROM users
)
SELECT user_id, name, status, updated_at
FROM ranked_users
WHERE rn = 1 AND status = 'Active';
```

Phase 2 — Regex extraction

The logs are loaded as a single text column. A vectorized string mask excludes records containing the word ERROR. Pandas str.extract() then captures four named groups: Date, User_ID, Product_ID, and EUR_Value. All 94,902 non-error records parsed successfully.

```
pattern = (
  r'\[(?P<Date>\d{4}-\d{2}-\d{2})\s+\d{2}:\d{2}:\d{2}\]'
  r'.*?"u":"(?P<User_ID>U-\d+)"'
  r'.*?"item_code":"(?P<Product_ID>PRD-\d+)"'
  r'.*?"eur_val":(?P<EUR_Value>\d+(?:\.\d+)?)'
)
extracted_data = valid_log_lines.str.extract(pattern)
```

Phase 3 — Currency and customer reconciliation

Exchange rates are sorted by date before applying forward-fill. The exchange-rate join uses a left join, preserving every parsed transaction and checking that a rate exists for each date. The final customer join is an inner join to retain only transactions linked to users whose latest account status is Active. USD_Revenue is calculated as EUR_Value multiplied by EUR_to_USD.

```
rates = rates.sort_values('Date')
rates['EUR_to_USD'] = rates['EUR_to_USD'].ffill()
with_rates = transactions.merge(rates, on='Date', how='left')
df_final = with_rates.merge(active_users,
    left_on='User_ID', right_on='user_id', how='inner')
df_final['USD_Revenue'] = (
    df_final['EUR_Value'] * df_final['EUR_to_USD'])
```

Phase 4 — Aggregation

The final dataset is grouped by calendar month to calculate total USD revenue. It is also grouped by customer to calculate total reconciled USD spending, sorted descending, and truncated to the top five customers. Matplotlib creates a monthly revenue line chart and a horizontal bar chart for the top five customers.

6. Results and Dashboard

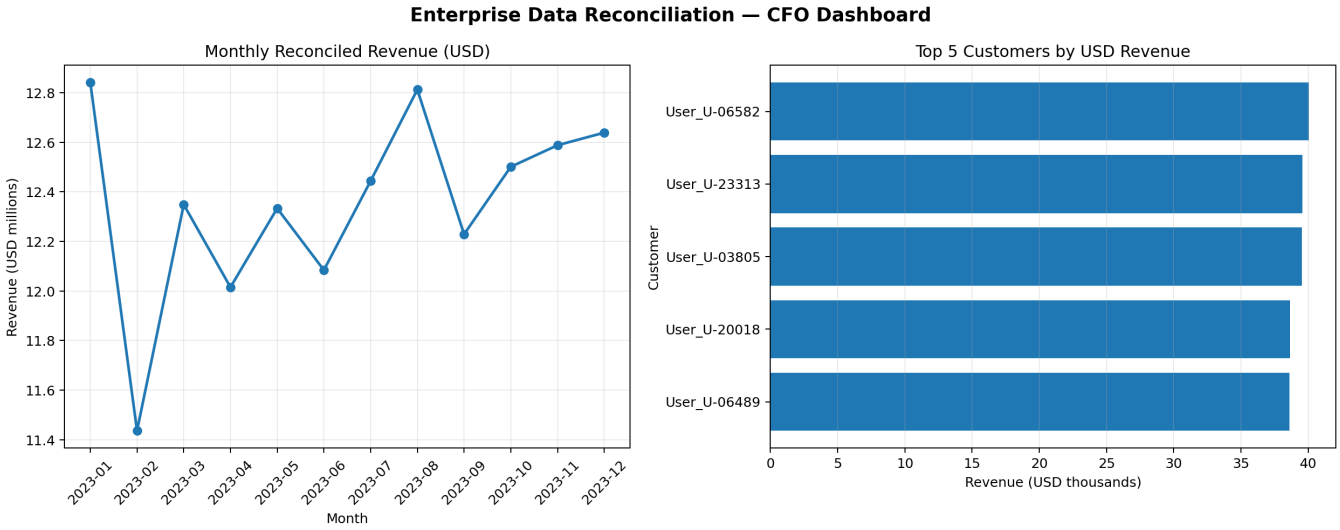


Figure 1. Monthly reconciled revenue and top five active customers by total USD revenue.

Monthly revenue table

Month	Revenue (USD)
2023-01	$12,841,761.38
2023-02	$11,436,130.90
2023-03	$12,348,931.84
2023-04	$12,015,376.19
2023-05	$12,333,644.45
2023-06	$12,084,355.82
2023-07	$12,444,163.58
2023-08	$12,812,730.66
2023-09	$12,228,952.05
2023-10	$12,501,330.36
2023-11	$12,588,223.09
2023-12	$12,639,022.60

Top five customers

User ID	Customer name	Total USD revenue
U-06582	User_U-06582	$40,050.99
U-23313	User_U-23313	$39,555.81
U-03805	User_U-03805	$39,529.95
U-20018	User_U-20018	$38,649.97
U-06489	User_U-06489	$38,629.14

7. Key Findings

• The dataset contained 100,000 raw log records; 5,098 error records were excluded.

• All 94,902 non-error records were successfully parsed into transaction fields.

• The SQL query selected 14,183 users whose latest account status was Active.

• The rate file had 104 missing rate values; chronological forward-fill removed the gaps.

• The active-user reconciliation produced 53,899 final transactions for 13,888 unique customers.

• Total reconciled revenue was $148,274,622.91.

• The highest revenue month was 2023-01 ($12,841,761.38); the lowest was 2023-02 ($11,436,130.90).

8. Limitations and Assumptions

• The final revenue total is limited to transactions matched to users whose latest status is Active; it is not the gross value of every successfully parsed transaction. • Forward-fill assumes the most recent available exchange rate remains applicable until a new rate is present. This is a standard business rule for this exercise, not an assertion about official market fixing methodology. • The supplied exchange-rate values are treated as the source of truth; this project does not independently verify their provenance. • The top-customer ranking uses the customer name stored in the user database and sums revenue across all reconciled transactions.

9. Conclusion

The project successfully combines SQL window functions, vectorized regex extraction, time-series gap handling, joins, and aggregation into a reproducible data analytics pipeline. It produced a reconciled transaction dataset, monthly revenue summaries, top-customer rankings, and a CFO dashboard. The processing time measured during this run was approximately 1.952 seconds, comfortably within the two-minute target described in the project statement.

10. Deliverables

The project folder contains the executable Python script, completed Jupyter notebook, README, source data files, CSV outputs, processing metrics, and dashboard image. The final reconciled dataset is stored at `output/final_reconciled_transactions.csv`.

Appendix — Output files

active_users.csv — Latest active customer records.

clean_transactions.csv — Parsed non-error transaction log rows.

final_reconciled_transactions.csv — Transactions reconciled to active customers and exchange rates.

monthly_revenue.csv — Monthly USD revenue aggregation.

top_5_customers.csv — Top five active customers by total USD revenue.

project_metrics.json — Counts, totals, and runtime metrics.

cfo_dashboard.png — Combined monthly revenue and top customer chart.